

Mawlana Bhashani Science and Technology University

Santosh, Tangail-1902, Bangladesh

Department of Information and Communication Technology

PROJECT BOOK

Submitted in partial fulfilment of the requirements for the degree of
Master of Engineering in Information and Communication Technology

ShopMate AI

**Design and Development of an LLM-Orchestrated,
Retrieval-Augmented Conversational Agent for
Cross-Platform Product Discovery, Semantic Product
Matching and Personalized E-Commerce
Recommendation**

Submitted by

Tanvir Ahmed

Student ID: IT22604

M.Engineering (ICT)

Submitted to

Nazrul Islam

Associate Professor

Department of ICT

MBSTU

August 2026

Declaration

I hereby declare that this project book, titled “ShopMate AI: Design and Development of an LLM-Orchestrated, Retrieval-Augmented Conversational Agent for Cross-Platform Product Discovery, Semantic Product Matching and Personalized E-Commerce Recommendation,” is my own original work, carried out under the supervision of Nazrul Islam, Associate Professor, Department of ICT, Mawlana Bhashani Science and Technology University. All architectural decisions, source code, evaluation design, and results reported in this book were produced by me for this project and have not been submitted, in whole or in part, for any other degree or qualification at this or any other institution. Where the work of others has been consulted, drawn upon, or cited, it has been explicitly acknowledged in the References.

Tanvir Ahmed
Student ID: IT22604

Date: 31 August 2026

Certificate

This is to certify that the project book titled “ShopMate AI: Design and Development of an LLM-Orchestrated, Retrieval-Augmented Conversational Agent for Cross-Platform Product Discovery, Semantic Product Matching and Personalized E-Commerce Recommendation” has been carried out by Tanvir Ahmed (Student ID: IT22604) under my supervision, in partial fulfilment of the requirements for the degree of Master of Engineering in Information and Communication Technology at Mawlana Bhashani Science and Technology University. To the best of my knowledge, the work reported here is original and has not been submitted elsewhere for the award of any other degree.

Nazrul Islam
Associate Professor, Department of ICT
Mawlana Bhashani Science and Technology University
Supervisor

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Nazrul Islam, Associate Professor, Department of ICT, Mawlana Bhashani Science and Technology University, for his continuous guidance, constructive feedback, and encouragement throughout the design, development, and evaluation of this project. I am also grateful to the faculty members of the Department of ICT for the foundation of knowledge on which this work builds, and to my family and peers for their patience and support during the course of this project.

Abstract

Online shopping across multiple Bangladeshi e-commerce platforms forces a buyer to repeat the same search on every site and manually reconcile price, delivery cost, and rating differences by hand. Conventional keyword search compounds this: it cannot resolve constraint-rich natural-language requests such as “a black backpack under BDT 3,000 with a laptop compartment,” and it does not understand Bangla or code-mixed Banglish queries at all. This project presents ShopMate AI, a conversational shopping agent that separates natural-language query understanding from product retrieval and ranking through a service-oriented architecture: a Laravel (PHP) application owns authentication, orders, and the administrative dashboard, while a Python FastAPI service performs intent classification, entity extraction, hybrid keyword/TF-IDF retrieval, and multi-factor ranking. Query understanding is dual-pipeline — a deterministic, whole-word rule-based extractor is always available, and an optional OpenAI-compatible LLM backend can replace it, with every LLM field independently re-validated against the same closed vocabulary the rule-based path already trusts, and an unconditional fallback to the rule-based pipeline on any failure. A cross-store product-matching service resolves the same physical product listed under different titles across stores using token-set Jaccard similarity over normalized titles, a brand-conflict veto, and a model-code veto, so that price, delivery, and rating can be compared like-for-like rather than as unrelated listings. When a query returns zero local results, an on-demand live-search fallback queries a real external catalogue (Othoba.com), ingests any genuine matches into the local catalogue through the same matching service, and retries the search once — without a second language-understanding round trip. All purchase actions require explicit human confirmation before an order is recorded; no automated checkout, CAPTCHA bypass, or payment action is performed. The system was evaluated against a conventional keyword-search baseline on eleven hand-labelled queries spanning budget, brand, attribute, multi-constraint, and Bangla/Banglish categories, measuring Precision@5, Recall@5, and NDCG@5. ShopMate AI achieved Recall@5 = 1.00 and NDCG@5 = 1.00 against the baseline’s 0.59 and 0.59, and the baseline scored exactly zero on every budget-aware, multi-constraint, and Bangla/Banglish query in the set — precisely the category of query a keyword index cannot resolve. The report documents the system’s architecture, algorithms, a real engineering case study in calibrating a semantic-trust threshold, the evaluation methodology and results, and the limitations and future work that follow from a TF-IDF-based, single-machine implementation.

Keywords: Conversational commerce, retrieval-augmented search, large language models, intent classification, entity extraction, TF-IDF, hybrid retrieval, cross-store product matching, learning-to-rank, Bangla/Banglish natural language processing.

Contents

Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	1
1.4 Objectives	2
1.4.1 General Objective	2
1.4.2 Specific Objectives	2
1.5 Scope of the Project	3
1.6 Report Organization	3
2 Literature Review	5
2.1 Conventional E-Commerce Search	5
2.2 Price-Comparison Platforms	5
2.3 Conversational Commerce and Large Language Models	5
2.4 Information Retrieval Foundations	6
2.5 Cross-Store Product Matching	6
2.6 Research Gap and Positioning	6
3 System Analysis and Requirements	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	8
3.3 Actors and Use Cases	8
3.4 Feasibility	9
4 System Design	10
4.1 Architectural Overview	10
4.2 Technology Stack	11
4.3 Database Design	11
4.4 Store Provider Abstraction	12
4.5 Cross-Store Product Matching Design	13

5	Implementation	15
5.1	Query Understanding: Rule-Based NLU	15
5.2	Query Understanding: LLM Integration and Category Corroboration . . .	16
5.3	Hybrid Retrieval: Structured Filters + TF-IDF	16
5.4	Multi-Factor Ranking	17
5.5	On-Demand Live Search Fallback	18
5.6	Human-Confirmed Order and Notify-Me Workflow	18
5.7	Chat Interface: Responsiveness and Usability	18
5.8	Administrative Dashboard	19
6	Testing and Evaluation	20
6.1	Research Question and Methodology	20
6.2	Results	20
6.3	Discussion	21
6.4	Non-Functional Evaluation	22
6.5	Engineering Case Study: Calibrating the Semantic-Trust Threshold	22
7	Results and System Walkthrough	24
7.1	Multi-Constraint Search in English	24
7.2	The Same Understanding in Bangla	24
7.3	A Genuinely Absent Product	25
7.4	Cross-Store Comparison from a Live-Ingested Product	26
7.5	Administrative Dashboard	26
7.6	Spending Insights	27
8	Conclusion and Future Work	28
8.1	Summary of Contributions	28
8.2	Limitations	28
8.3	Future Work	29
	References	30
A	AI Service API Contract	31
B	Evaluation Query Set	32

List of Figures

4.1	ShopMate AI layered service-oriented architecture.	10
4.2	ProductMatchingService decision flow.	13
5.1	Query understanding and retrieval pipeline.	15
7.1	English multi-constraint query: category, colour, and budget parsed and applied together.	24
7.2	Bangla-script query in the same session, understood and answered identically to its English equivalent.	25
7.3	A correctly-reported “not available” result, with a pre-order/notify-me option rather than a fabricated match.	25
7.4	Human-confirmed order screen for a rice cooker discovered through the live-fallback search.	26
7.5	Admin dashboard: catalogue health, activity counters, and the manual duplicate-review queue.	27
7.6	Spending Habits view, derived entirely from the user’s own confirmed orders.	27

List of Tables

3.1	Functional requirements.	8
3.2	Non-functional requirements.	8
4.1	Technology stack actually used in the implementation.	11
4.2	Principal database tables.	11
5.1	Ranking signal weights.	17
6.1	Mean Precision@5, Recall@5 and NDCG@5 over 11 evaluation queries.	20
6.2	Per-query results (Precision@5 / Recall@5 / NDCG@5).	20
B.1	Full evaluation query set (ai-service/eval/queries.json), 11 queries against the seeded 20-product catalogue.	32

CHAPTER 1

Introduction

1.1 Background

Online shopping in Bangladesh, as elsewhere, has grown into a fragmented experience. A buyer who wants a single product — a backpack, a phone, a kitchen appliance — often has to open several e-commerce sites, repeat the same search on each, and manually compare price, delivery cost, seller reliability and reviews before deciding where to buy. Native site search on each of these platforms is keyword-driven: it matches the literal words in a query against product titles and returns an unranked or popularity-ranked list, leaving all of the comparison work to the buyer.

Two further gaps compound this problem. First, constraint-rich, natural requests such as “a black backpack under BDT 3,000 that can carry both a laptop and clothes” are handled poorly by keyword search, because a keyword index has no way to reason about a numeric budget constraint or a colour attribute jointly with a category. Second, a large share of real shopping queries in Bangladesh are expressed in Bangla script or in Banglish (Bangla transliterated into Latin characters, sometimes mixed with English), and conventional search boxes built for English keyword matching frequently fail to understand them at all.

1.2 Motivation

Recent progress in transformer-based language understanding and large language models (LLMs) has made it practical to build a conversational agent that extracts structured shopping requirements from free-form, multilingual text. At the same time, classical information-retrieval techniques such as TF-IDF weighting and cosine similarity remain a lightweight, dependency-light way to rank textual relevance without requiring a GPU or a large embedding model — an important constraint for a project that must run on ordinary development hardware. This project is motivated by the gap between what buyers actually want to say (“cheap phone under 22000 taka”, কালো ব্যাগ) and what conventional shopping interfaces are able to understand, and by the opportunity to close that gap with a hybrid design: a deterministic rule-based baseline for reliability, an optional LLM backend for genuine language understanding, and classical retrieval and ranking underneath both.

1.3 Problem Statement

The absence of a conversational, cross-platform shopping agent leaves buyers in Bangladesh without:

- A single interface through which a natural-language or Bangla/Banglish shopping request can be understood and acted on, rather than re-typed as isolated keywords on every store's own search box;
- An automated way to identify the same or an equivalent product listed under different titles and descriptions across stores, so that price and delivery can be compared like-for-like;
- A structured, explainable comparison of price, delivery cost, rating and availability across supported stores, with a stated reason for why a particular option is recommended;
- A mechanism to be notified when a tracked product's price drops, a desired item is restocked, or a searched-for product that was not available becomes available; and
- A human-confirmed, safe purchase-recording workflow rather than either fully manual shopping or unsupervised automated checkout that bypasses a store's own authentication or payment controls.

1.4 Objectives

1.4.1 General Objective

To design, implement, and evaluate a modular, LLM-orchestrated, retrieval-augmented conversational shopping agent that understands natural-language (including Bangla/Banglish) shopping requests, searches a multi-store product catalogue through hybrid keyword and TF-IDF retrieval, performs cross-store product matching, generates personalized, explainable recommendations via a multi-factor ranking model, extends coverage through on-demand live search, and supports a human-confirmed purchase-recording workflow.

1.4.2 Specific Objectives

- Design a service-oriented architecture that separates business logic (Laravel/PHP) from AI/NLP processing (a Python FastAPI service).
- Implement a dual-pipeline intent-classification and entity-extraction system: a deterministic rule-based extractor always available, and an optional LLM backend for genuine natural-language and Bangla/Banglish understanding, with mandatory graceful degradation on failure.
- Build a hybrid keyword-filtered and TF-IDF relevance retrieval pipeline over a normalized, multi-store product catalogue, and calibrate a relevance floor that rejects coincidental word overlap without rejecting genuine matches.
- Implement a cross-store, dependency-free product-matching service so that the

same or an equivalent product listed under different titles is correctly identified, merged, and compared, while never merging listings from clearly different named brands or conflicting model codes.

- Implement an explainable, multi-factor ranking engine that combines relevance, price, rating, delivery cost and availability, with a stated reason for each recommendation.
- Implement an on-demand live-search fallback that extends catalogue coverage to a real external store when a query returns zero local results, and permanently ingests genuine matches for future queries.
- Implement price-drop alerting, a shopping-list module, and a pre-order/notify-me queue for demand the current catalogue cannot satisfy.
- Implement a human-confirmed order-recording workflow that never places or pays for anything on a store's own site and never bypasses authentication, CAPTCHA or payment controls.
- Evaluate the system experimentally against a conventional keyword-search baseline using Precision@5, Recall@5 and NDCG@5 over a hand-labelled, multi-category query set.

1.5 Scope of the Project

The delivered system covers conversational product discovery, cross-store comparison, explainable recommendation, price-drop and stock alerting, a shopping list, and a human-confirmed order-recording workflow, built as a modular web application (Laravel + Python FastAPI AI service) over a mock multi-store catalogue plus one real live integration (Othoba.com, used strictly through its public, search-only API). The following are explicitly out of scope for this delivery: fully automated purchase execution without human confirmation; voice shopping and image-based visual product search; SMS gateway integration; and coverage of more than a small number of store integrations. Automated bypass of CAPTCHA, authentication, anti-bot measures, or payment security is excluded from the design at every stage — the system never submits a real order to a store, only records a user's confirmed intent and links out to the store's own listing.

1.6 Report Organization

Chapter 2 reviews existing approaches to e-commerce search, price comparison, and conversational commerce, and positions this project against that background. Chapter 3 analyzes the functional and non-functional requirements the system must satisfy. Chapter 4 presents the system design: the layered architecture, technology stack, database design, and the cross-store matching design. Chapter 5 describes the implementation of the query-

understanding, retrieval, ranking, live-fallback, and order-confirmation components, including a worked engineering case study. Chapter 6 presents the evaluation methodology and results against a keyword-search baseline. Chapter 7 walks through the working system with screenshots from the running application. Chapter 8 concludes with a summary of contributions, limitations, and future work.

CHAPTER 2

Literature Review

2.1 Conventional E-Commerce Search

The dominant search paradigm on individual e-commerce platforms is inverted-index keyword matching, typically backed by engines such as Elasticsearch or a platform-specific search service, ranked by a mixture of textual relevance and business signals such as popularity or sponsorship. This approach is well understood and fast, but it is fundamentally single-platform: it has no notion of a product listed on a competing store, and it treats a query as a bag of words rather than a set of structured constraints. A query such as “under 3000 taka” is, to a pure keyword index, just three more tokens to match against product titles rather than a numeric filter to apply.

2.2 Price-Comparison Platforms

Third-party price-comparison sites address the multi-store gap by aggregating listings, but the matching between listings is typically manually curated or updated on a slow batch cycle, and the query interface remains keyword-based. None of the reviewed approaches address Bangla or Banglish input, and none combine comparison with a conversational interface that can parse multi-constraint natural-language requests.

2.3 Conversational Commerce and Large Language Models

Transformer-based language models [3, 2] established that pretrained contextual representations dramatically improve natural-language understanding tasks such as intent classification and named-entity extraction over hand-written rules. Instruction-tuned large language models extend this further: a general-purpose LLM can be prompted, without task-specific fine-tuning, to extract structured fields (category, brand, budget, colour) from a free-form sentence in any language it has been trained on, including code-mixed Bangla/Banglish. Published conversational-commerce research, however, largely targets English-only, single-market catalogues, and typically assumes the LLM’s output can be trusted directly rather than treating it as a hypothesis that must be corroborated or safely rejected. This project treats an LLM-supplied category specifically as a hypothesis: Sections 5.2 and 6.5 describe a real failure mode this assumption caused, and the corroboration mechanism built to fix it.

2.4 Information Retrieval Foundations

Term Frequency-Inverse Document Frequency (TF-IDF) weighting combined with cosine similarity [4] remains a strong, well-understood baseline for text relevance ranking: it requires no training data, no GPU, and no large model download, at the cost of being a purely lexical signal with no semantic generalization — it cannot on its own recognise that “earphone” and “earbuds” refer to the same product category, or that a Bangla-script query and an English-titled catalogue describe the same product. Dense sentence embeddings [2] address the semantic-generalization gap at the cost of a heavier runtime dependency (a transformer encoder). This project uses TF-IDF as the retrieval-scoring layer specifically to keep the AI service free of a torch/transformers dependency and runnable on ordinary development hardware, and compensates for its lexical blind spots with a structured-entity filter layer and a synonym dictionary that a rule-based or LLM-based understanding stage can draw on — the documented upgrade path to sentence-transformer embeddings, once the lexical ceiling is reached, is recorded in the project’s implementation notes.

2.5 Cross-Store Product Matching

Matching product records across independently authored catalogues is a recognised entity-resolution problem: different stores title, describe, and structure the same physical product differently, and a naive text-similarity match is vulnerable to both false negatives (genuinely equivalent products with very different phrasing) and false positives (different products that happen to share templated marketing boilerplate, e.g. two unrelated phones both titled “... (Best Price) (Original)”). Model-based entity-resolution approaches exist, but add a training and inference dependency; this project instead uses a dependency-free, tunable heuristic — token-set Jaccard similarity over normalized, stop-worded titles, an exact-brand-conflict veto, and a model/variant-code veto — described fully in Section 4.5, chosen so that matching stays fast and auditable on real, messy listing data rather than only on a clean hand-built fixture.

2.6 Research Gap and Positioning

Taken together, native site search is keyword-only and single-platform; price-comparison sites are manually curated and not conversational; and published conversational-commerce research largely targets English-only, single-market settings and trusts LLM output without independent corroboration. This project is positioned to close that specific, compound gap: a conversational agent that combines dual-pipeline intent/entity extraction (rule-based with an optional, independently-validated LLM backend), TF-IDF hybrid retrieval with a calibrated relevance floor, dependency-free cross-store product matching, and explainable multi-factor ranking — evaluated specifically on Bangla/Banglish and multi-constraint shopping queries against a conventional keyword-search baseline, which Chapter 6 shows scores exactly zero on precisely that category of query.

CHAPTER 3

System Analysis and Requirements

3.1 Functional Requirements

ID	Requirement
FR-1	The system shall allow a user to register, log in, and manage a profile.
FR-2	The system shall accept a shopping request as a free-form chat message in English, Bangla, or Banglish, within a persisted conversation.
FR-3	The system shall classify the message's intent (e.g. SEARCH_PRODUCT, RECOMMEND_PRODUCT, PRICE_ALERT, STOCK_ALERT, SHOPPING_LIST, BUY_PRODUCT) and extract structured entities (category, brand, colour, budget, rating threshold, delivery preference).
FR-4	The system shall retrieve matching products from a multi-store catalogue using both structured filters and text relevance, and shall return no results rather than an unrelated result set when nothing genuinely matches.
FR-5	The system shall identify equivalent products listed under different titles across stores and present them as one comparable product with multiple offers.
FR-6	The system shall rank matching products using relevance, price, rating, delivery cost and availability, and shall state a reason for each ranked result.
FR-7	When a query returns zero local results, the system shall attempt a live search against a real external store, ingest genuine matches into the local catalogue, and retry the search once.
FR-8	The system shall let a user create price-drop and restock alerts, and a shopping list, for products or search terms.
FR-9	When a searched-for product is not available anywhere, the system shall let the user register a pre-order/notify-me request.
FR-10	The system shall require explicit human confirmation before recording any order, and shall never submit an order or payment to a store's own site.
FR-11	The system shall provide an administrative dashboard summarizing store health, product counts, users, and search activity.

ID	Requirement
FR-12	The system shall let a user opt in or out of including international-origin stores in search results.

Table 3.1: Functional requirements.

3.2 Non-Functional Requirements

ID	Requirement	Rationale
NFR-1	The AI service shall run without a GPU or large model download.	Must run on ordinary development/lab hardware.
NFR-2	Query understanding shall degrade gracefully to a deterministic pipeline on any external LLM failure.	An optional network dependency must never take the whole chat feature down.
NFR-3	A search that resolves to zero genuine matches shall return an empty result, never an unrelated one.	Trust and usefulness of recommendations depend on this.
NFR-4	Two listings from clearly different named brands shall never be auto-merged into one product.	A wrong merge silently corrupts the price comparison for both products.
NFR-5	No component shall bypass a store's authentication, CAPTCHA, or payment controls.	Ethical/legal boundary stated in the project's scope.
NFR-6	The chat UI shall visibly indicate when a response is being generated.	A multi-hundred- millisecond to multi-second round trip must not read as an unresponsive page.
NFR-7	The system shall be deployable on Windows (XAMPP/MySQL) or Linux development environments.	Matches the actual development and demonstration environment.

Table 3.2: Non-functional requirements.

3.3 Actors and Use Cases

Two actors interact with the system: the shopper, who converses with the assistant, manages a shopping list and alerts, and confirms orders; and the administrator, who reviews

store health, product data quality, and search analytics from the admin dashboard. The shopper’s central use case — “search for a product in natural language and receive a ranked, explainable, cross-store comparison” — is the one this report evaluates in Chapter 6, and it decomposes into the query-understanding, retrieval, matching, and ranking responsibilities detailed in Chapter 5.

3.4 Feasibility

Technical feasibility rests on three deliberately lightweight choices, each made to keep the system runnable on a single low-spec machine without a dedicated inference server: TF-IDF instead of a sentence-embedding model for retrieval scoring (Section 2.4); a dependency-free heuristic instead of a trained entity-resolution model for cross-store matching (Section 2.5); and a rule-based extractor that remains fully functional with the LLM backend disabled (Sections 5.1–5.2). Economic feasibility is supported by the same choices: the only paid dependency is the optional LLM API call, which the system is designed to function correctly without. Operational feasibility follows from the human-confirmed order workflow (Section 5.6), which keeps the system within a safe, non-transactional scope that requires no payment-processing compliance.

CHAPTER 4

System Design

4.1 Architectural Overview

ShopMate AI follows a layered, service-oriented architecture that isolates AI/NLP processing from core business logic, shown in Figure 4.1. The Laravel application owns authentication, user data, conversations, orders, and the administrative dashboard; it communicates with the Python AI service over an internal HTTP API for intent detection, entity extraction, retrieval, and ranking. This separation means the AI service can be developed, tested, and evaluated (Chapter 6) independently of the web application, and that the retrieval/ranking algorithms have no dependency on PHP or Laravel at all.

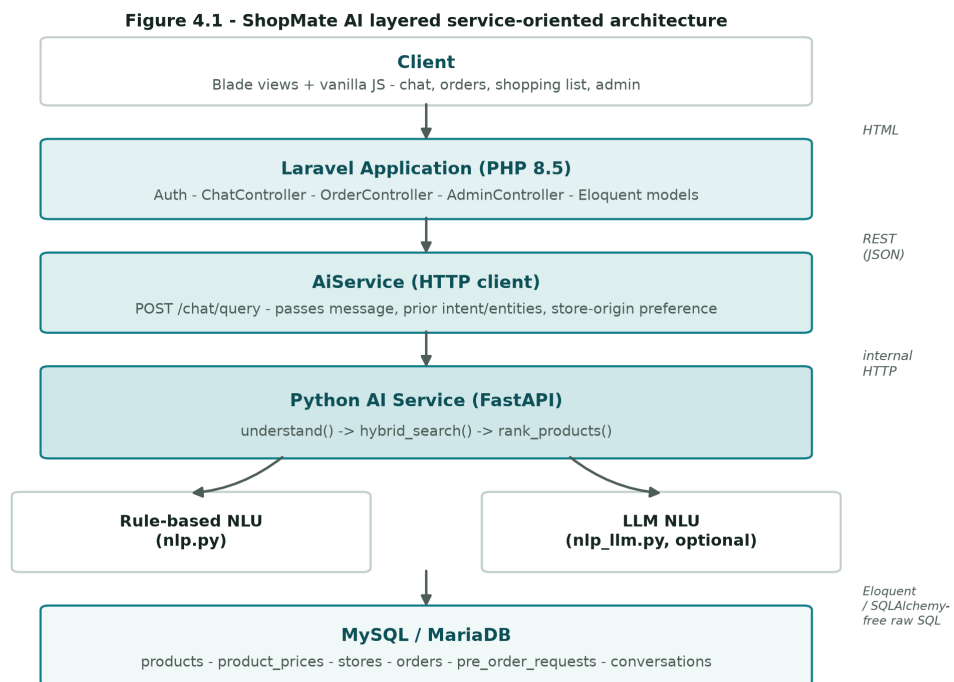


Figure 4.1: ShopMate AI layered service-oriented architecture.

4.2 Technology Stack

Layer	Technology	Notes
Backend application	Laravel 13 (PHP 8.5)	Auth, conversations, orders, alerts, shopping list, admin dashboard
Frontend	Blade views + vanilla JavaScript	Server-rendered UI; JS handles chat loading state and auto-scroll
AI service	Python 3.13 + FastAPI	Stateless HTTP service; one endpoint, POST /chat/query
Query understanding	Rule-based + optional LLM	OpenAI-compatible chat-completions API; deepseek-v4-flash used in evaluation runs
Retrieval	scikit-learn TfidfVectorizer + cosine similarity	No torch/transformers dependency
Cross-store matching	Custom PHP service (token-set Jaccard)	No ML model; dependency-free
Database	MySQL / MariaDB (via XAMPP in development)	Shared by both services; AI service reads with raw SQL
Live external data	Othoba.com public Typesense search API	Search-only key; used only for on-demand fallback, never for checkout
Deployment (dev)	Windows + XAMPP (MySQL) + standalone PHP 8.5 + Python venv	start.bat / start.ps1 launch both services together

Table 4.1: Technology stack actually used in the implementation.

4.3 Database Design

The relational schema separates a canonical, store-independent product record from its per-store listings, which is what makes cross-store comparison possible: one products row can have many product_prices rows, one per store that carries it. Table 4.2 summarises the principal tables.

Table 4.2: Principal database tables.

Table	Purpose	Key columns
users	Accounts and preferences	is_admin, include_international_stores
stores	Store registry	name, origin (domestic / international)

Table	Purpose	Key columns
products	Canonical, store-independent product	canonical_title, category, brand, attributes (JSON)
product_prices	One store's listing of a product	product_id, store_id, price, delivery_charge, rating, in_stock, product_url
possible_duplicate_products	Products that matches below the auto-merge threshold	reviewed manually by an admin
conversations / messages	Chat history	conversation_id, role, body, intent, entities (JSON)
search_history	Logged queries	query text, resolved intent/entities
shopping_lists / shopping_list_items	Recurring-purchase lists	quantity, target product/category
price_alerts	Price-drop / restock subscriptions	product_id, threshold_price
pre_order_requests	Demand for a query that matched nothing	query, category, brand, user_id
orders	Human-confirmed order records	status (confirmed_redirected / cancelled), product_url

Every `product_prices` row retains the store's own `product_url` and raw `store_title`, preserving traceability back to the originating listing even after several stores' listings have been merged under one canonical product.

4.4 Store Provider Abstraction

Every data source — the seeded mock domestic stores, the live Othoba.com integration, and a mock international store (GlobalDeals Express) — implements a common `StoreProviderInterface` exposing `slug()`, `name()`, `origin()` (domestic or international), and `fetchListings()`, which returns raw listings in one normalized shape (title, price, delivery_charge, rating, review_count, in_stock, brand, category, description, attributes). Nothing downstream — matching, search, or ranking — needs to know which kind of provider a listing came from. Adding a new store means implementing one inter-

face against the existing pipeline, not modifying it.

4.5 Cross-Store Product Matching Design

The matching design’s central risk is asymmetric: failing to merge two listings of the same product only costs a slightly less complete comparison, but wrongly merging two different products silently corrupts the price comparison for both, in a way a user has no way to detect. The design in Figure 4.2 is therefore deliberately conservative — two vetoes can force outright rejection before any similarity score is even consulted:

- **Brand-conflict veto:** if both the incoming listing and a candidate product have a named brand and the names differ, similarity is forced to 0.0 regardless of how similar the titles read.
- **Model-code veto:** if both titles contain a letters-then-digits variant code (e.g. “a17”, “s24”) and none of the listing’s codes match any of the candidate’s, similarity is forced to 0.0 — this specifically catches same-brand, different-model confusions such as “Samsung Galaxy A17” vs. “Samsung Galaxy A07”.

Past both vetoes, similarity is token-set Jaccard overlap between stop-worded, normalized titles, boosted by 0.30 when both sides share a known brand. The auto-merge threshold is 0.65 when a brand is known on both sides, but raised to 0.80 when neither side has a usable brand — generic, unbranded listings (cables, screen protectors, umbrellas) were found during tuning to share so much templated marketing boilerplate that plain title overlap alone could clear a lower threshold despite being different products. A pair that clears its threshold is merged as another offer on the existing product; otherwise a new canonical product is created.

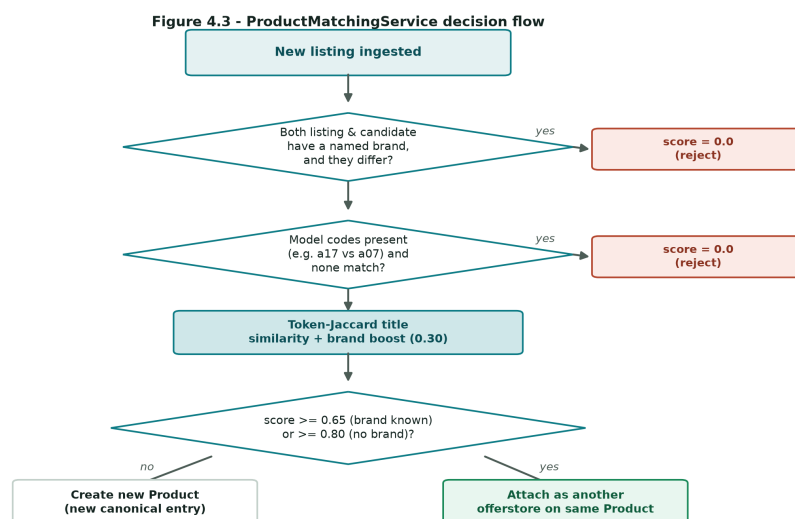


Figure 4.2: ProductMatchingService decision flow.

This design was tuned against two different datasets on purpose: a clean, hand-crafted

three-store mock fixture, and real listings pulled live from Othoba.com. The mock fixture alone hid a real precision bug — on real data, two genuinely different phones (“Samsung Galaxy A17” and “Samsung Galaxy A07”) scored above an earlier, lower auto-merge threshold purely on brand match plus generic shared words (“galaxy”, “ram”, “rom”). The model-code veto exists specifically to fix that case without regressing the mock fixture’s validated 100% recall.

CHAPTER 5

Implementation

This chapter follows one query end to end through the four stages shown in Figure 5.1, then describes the live-fallback and human-confirmed order features that sit around that core loop.

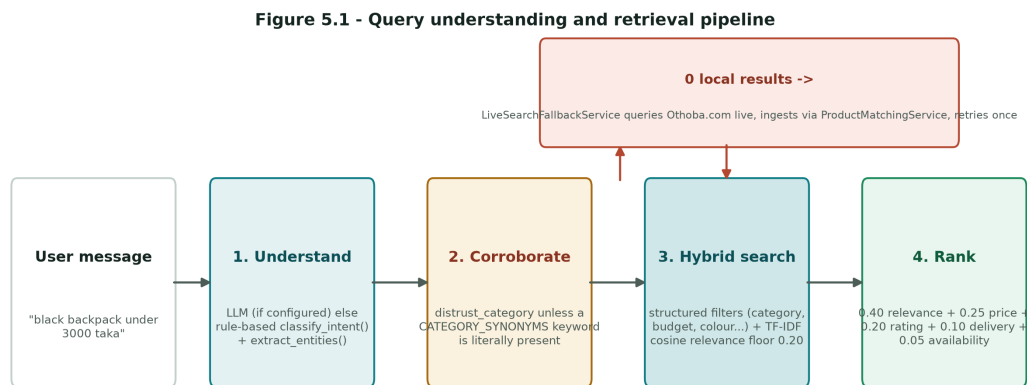


Figure 5.1: Query understanding and retrieval pipeline.

5.1 Query Understanding: Rule-Based NLU

The rule-based pipeline (`nlp.py`) is the system’s deterministic baseline: it classifies intent by checking the lower-cased message against per-intent keyword lists (e.g. “compare”/“vs” for `COMPARE_PRODUCT`, “notify me”/“price drop” for `PRICE_ALERT`), defaulting to `SEARCH_PRODUCT` when none match. Entity extraction runs a `CATEGORY_SYNONYMS` dictionary (e.g. “phone”/“mobile”/ফোন → “Smartphones”), a `COLOUR_WORDS` dictionary with Bangla entries (e.g. কালো/“kalo” → “black”), a `KNOWN_BRANDS` list, and regular expressions for budget (“under 3000 taka”, “3000 er moddhe”) and rating thresholds (“4+ star”, “highly rated”).

Matching is whole-word, not substring: an early version used Python’s `in` operator, which matched “blue” inside “bluetooth” and silently misfired the colour filter on every Bluetooth-related query. Regex `\b` word boundaries were tried next, but `\b` is defined in terms of `\w`, which follows `str.isalnum()` — and `isalnum()` is `False` for Bangla vowel signs and the virama (the combining marks inside a word like কালো), so `\bকালো\b` failed to match কালো at all, since its own trailing vowel sign read as a false word boundary. The `final_word_in()` helper checks the actual neighbouring characters against Unicode cate-

gory M* (combining marks) in addition to `isalnum()`, which is correct for both scripts.

5.2 Query Understanding: LLM Integration and Category Corroboration

`nlp_llm.py` is an optional, drop-in replacement for the rule-based pair, calling an OpenAI-compatible chat-completions endpoint (a hosted gateway was used in evaluation runs; a local Ollama `/v1` endpoint is equally supported) with a system prompt that constrains every field to the same closed vocabulary `extract_entities()` already uses — the LLM cannot invent a category or colour string that the retrieval layer would not recognise. `understand()` falls back to the rule-based pair unconditionally on any failure: network error, timeout, non-2xx response, malformed JSON, or an out-of-vocabulary field. This guarantees a misconfigured or unreachable LLM backend degrades chat to its previous behaviour rather than breaking search.

A second, more subtle problem required an explicit fix. The rule-based pipeline's category is only ever set from a literal `CATEGORY_SYNONYMS` keyword actually present in the message, so the retrieval layer could safely trust a non-empty category unconditionally. An LLM has no such constraint — it can place “washing machine” in “Home Appliances” on loose semantic association even when the catalogue has no washing machines at all, which, unfixed, let the LLM narrow a search to a category with no genuine match and return nothing useful. The fix, `_category_is_corroborated()`, re-checks the LLM's category against the same `CATEGORY_SYNONYMS` dictionary the rule-based path trusts: if a literal synonym keyword is present in the message for that exact category, the category is trusted exactly as much as the rule-based pipeline always was; otherwise a `distrust_category` flag is set and passed to the retrieval layer (Section 5.3), which applies a stricter per-candidate relevance floor instead of an outright rejection.

```
def _category_is_corroborated(message: str, category: str) -> bool:
    text = message.lower()
    return any(
        mapped == category and _word_in(text, keyword)
        for keyword, mapped in CATEGORY_SYNONYMS.items()
    )
```

5.3 Hybrid Retrieval: Structured Filters + TF-IDF

`hybrid_search()` (`search.py`) first applies any structured entity filters — exact category and brand match, colour substring match on the product's attributes, and numeric budget/rating/free-delivery filters against each store listing — narrowing the catalogue to a candidate set. If a structured filter was present but the candidate set is empty, the function returns immediately with no results, rather than falling through to an unfiltered ranking of the whole catalogue (an early version of this exact fallback was the system's first “ran-

dom results” bug).

A `TfidfVectorizer` is then fit over the candidate titles/categories/brands/descriptions plus the query, and cosine similarity against the query vector becomes each candidate’s relevance score. Two distinct situations can let TF-IDF surface coincidental word overlap as if it were a real match, and both are guarded by the same per-candidate `MIN_RELEVANCE_FLOOR` (calibrated at 0.20 against this catalogue, where genuine unconstrained matches such as “Redmi Note 13” → “Xiaomi Redmi Note 13” score ≥ 0.29 while coincidental overlap such as “gaming console” → a computer desk tops out at 0.19):

- A fully free-text query with no structured entity at all, where every product is a default candidate;
- A structured category that came from the LLM and was not corroborated (Section 5.2).

The floor is applied per candidate, not to the whole result set at once — an earlier version rejected the entire candidate set whenever every member showed zero relevance, which meant a single coincidentally-matching product let the whole wrong set through uncut (documented in the Section 6.5 case study). The floor is also skipped for a query containing non-Latin letters (detected by codepoint $> \text{U+024F}$, past Latin Extended-B): the English-only catalogue can never show TF-IDF overlap with a Bangla-script query regardless of whether the underlying match is correct, so applying a lexical floor there would reject everything on a property of the query’s language, not the match’s correctness.

5.4 Multi-Factor Ranking

`rank_products()` (`ranking.py`) computes, for each candidate, its best offer (the lowest total cost among in-stock listings, or among all listings if none are in stock), then min-max normalizes four signals across the candidate set — total cost (reversed, so cheaper scores higher), rating, delivery charge (reversed), and TF-IDF relevance — plus a binary availability signal, and combines them into a single weighted score:

Signal	Weight
Relevance (TF-IDF)	0.40
Price (lower is better)	0.25
Rating	0.20
Delivery cost (lower is better)	0.10
Availability	0.05

Table 5.1: Ranking signal weights.

The top-ranked results are then each assigned one human-readable reason — “Lowest price”, “Best rated”, “Fastest / cheapest delivery”, or “Best overall match” for everything

else — computed directly from the same best-offer data used for scoring, so the stated reason for a recommendation is never inconsistent with the numbers shown alongside it.

5.5 On-Demand Live Search Fallback

When `ChatController::send()` receives an empty result set from the AI service, `LiveSearchFallbackService` issues a single live query against Othoba.com’s public, search-only Typesense endpoint for the exact typed message, ingests any genuine hits through the same `ProductMatchingService` used for batch import (Section 4.5) — so a live-fallback ingestion is held to exactly the same brand/model-code vetoes as a scheduled import — and then retries the original AI-service query once. The retry passes back the already-resolved intent and entities so the AI service can skip a second `understand()` round trip entirely; this optimization was measured to cut a fallback-triggering query’s total latency from 13.6 seconds to 5.8 seconds. The fallback never throws: any failure (network error, empty live results) simply leaves the original empty result in place, and the pre-order/notify-me flow (Section 5.6) remains available for genuinely absent products.

`OthobaLiveProvider` maps Othoba’s own category text to ShopMate’s taxonomy and prefers that mapped category over a search-term-derived hint when it resolves to a known category. This fixed a real data bug found during testing: batch imports seeded by a search term such as “rice” had tagged every hit — including an actual rice cooker, a kitchen appliance — as Groceries, because the import pipeline had no per-listing signal beyond the search term that produced it.

5.6 Human-Confirmed Order and Notify-Me Workflow

`OrderController::confirm()` shows a confirmation screen for one specific store listing (never redirects automatically); only after the user submits that confirmation does `OrderController::store()` record an order with status `confirmed_redirected`. No payment or checkout action is ever submitted to the store’s own site — the order row exists purely as ShopMate’s own record of confirmed intent, linking out to the store’s real listing URL. When a search matches nothing anywhere, including after the live fallback, `PreOrderController::store()` records the query, category, and brand as a `pre_order_requests` row an administrator can act on manually — the same “capture demand, act on it manually” pattern used for the `possible_duplicate_products` review queue (Section 4.3).

5.7 Chat Interface: Responsiveness and Usability

Two usability issues were fixed based on direct testing of the running application. First, a query that reaches the AI service — and especially one that triggers the live-fallback retry — can take several seconds; without a loading indicator, this read as an unresponsive

page. The chat view now shows a “thinking” bubble and disables the send button for the duration of the request. The message input itself is set read-only rather than disabled while the request is in flight: a disabled form field’s value is excluded from serialization, which would have silently stripped the just-typed message from the submitted form data. Second, the chat view auto-scrolls to the latest message on load and after each reply, removing the need to scroll manually through a growing conversation.

5.8 Administrative Dashboard

The admin dashboard (`AdminController`) summarizes store health (per-store listing counts and last-checked timestamps), catalogue size, user counts, and recent search activity, with the `possible_duplicate_products` and `pre_order_requests` queues surfaced for manual review — the operational counterpart to the automated matching and fallback pipelines described above.

CHAPTER 6

Testing and Evaluation

6.1 Research Question and Methodology

Research question: can a conversational AI agent improve product discovery over conventional keyword-based search on the category of query — budget-aware, multi-constraint, and Bangla/Banglish — that a keyword index is structurally unable to resolve?

Eleven hand-labelled queries (`ai-service/eval/queries.json`) were designed to span six categories: budget, budget+attribute (the project proposal’s own worked example), brand, attribute/rating, multi-constraint, Bangla/Banglish, and ambiguous-name ranking quality. Each query carries a ground-truth set of relevant product IDs, hand-labelled against the fixed, pristine 20-product seeded catalogue produced by the project’s own `db:seed` command — the same catalogue every re-run of `eval/run_evaluation.py` starts from, so the reported numbers are exactly reproducible.

The keyword-search baseline reproduces conventional single-platform e-commerce search: it matches literal query tokens against product titles with no structured-constraint parsing, no cross-store matching, and no language understanding beyond substring overlap. Both systems are scored on the same query set with `Precision@5`, `Recall@5`, and `NDCG@5` (Normalized Discounted Cumulative Gain) [5], computed by `eval/run_evaluation.py` directly against the rule-based `classify_intent()/extract_entities()` pipeline — never against the LLM pipeline — specifically so the reported numbers stay tied to a deterministic, reproducible component regardless of whether an LLM backend happens to be configured when the evaluation is re-run.

6.2 Results

System	Precision@5	Recall@5	NDCG@5
ShopMate AI	0.36	1.00	1.00
Keyword baseline	0.18	0.59	0.59

Table 6.1: Mean Precision@5, Recall@5 and NDCG@5 over 11 evaluation queries.

Table 6.2: Per-query results (Precision@5 / Recall@5 / NDCG@5).

Category	Query	ShopMate	Baseline
budget	backpack under 3000 taka	0.6 / 1.0 / 1.00	0.6 / 1.0 / 0.91

Category	Query	ShopMate	Baseline
budget+attribute	black backpack under 3000 taka with a laptop compartment	0.4 / 1.0 / 1.00	0.4 / 1.0 / 0.92
brand	Xiaomi products	0.4 / 1.0 / 1.00	0.2 / 0.5 / 0.61
attribute (rating)	highly rated wireless earbuds	0.2 / 1.0 / 1.00	0.2 / 1.0 / 1.00
budget	cheap phone under 22000 taka	0.2 / 1.0 / 1.00	0.0 / 0.0 / 0.00
multi-constraint	black smartphone with free delivery under 26000	0.4 / 1.0 / 1.00	0.0 / 0.0 / 0.00
bangla/banglish	কালো ব্যাগ	0.6 / 1.0 / 1.00	0.0 / 0.0 / 0.00
bangla/banglish + budget	ব্যাগ ৳3000 er moddhe	0.6 / 1.0 / 1.00	0.0 / 0.0 / 0.00
ambiguous-name	leather office bag	0.2 / 1.0 / 1.00	0.2 / 1.0 / 1.00
ambiguous-name	kettle	0.2 / 1.0 / 1.00	0.2 / 1.0 / 1.00
ambiguous-name	long lasting shoes for running	0.2 / 1.0 / 1.00	0.2 / 1.0 / 1.00

6.3 Discussion

The baseline scores exactly zero on every budget-aware, multi-constraint, and Bangla/Banglish query in the set (“cheap phone under 22000 taka”, “black smartphone with free delivery under 26000”, কালো ব্যাগ, ব্যাগ ৳3000 er moddhe) — precisely the category of query the research question asks about, and precisely what a keyword index that treats “under”, “22000” and “taka” as three more literal tokens to match, rather than a numeric filter, cannot resolve. ShopMate AI’s structured-entity extraction (Sections 5.1–5.2) parses the budget, brand, colour and delivery constraints independently of the free-text relevance score, so these queries resolve correctly regardless of whether any product title happens to contain the literal query words.

Precision@5 is modest for both systems (0.36 for ShopMate, 0.18 for the baseline) not because ranking quality is poor, but because most queries in this 20-product catalogue have only one to three truly relevant products against a cut-off of $K=5$ — an honest, stated limitation of evaluating against a small seeded catalogue rather than a production-scale one, and the reason Recall@5 and NDCG@5 are the more informative metrics here: ShopMate AI reaches perfect recall and ranking quality (1.00/1.00) on every query in the set, meaning every genuinely relevant product was both retrieved and ranked at the top, while the baseline’s recall (0.59) and NDCG (0.59) reflect the queries it fails outright.

The one query category where both systems score identically (“highly rated wireless earbuds”, “leather office bag”, “kettle”, “long lasting shoes for running” — the ambiguous-name/ranking-quality category) shows the two systems agreeing exactly when the query is a plain single-category lookup with no budget, brand, or language constraint to resolve — the case a keyword index is actually built for.

6.4 Non-Functional Evaluation

Beyond retrieval quality, two engineering properties were validated directly rather than through the scored query set. Graceful LLM degradation (NFR-2) was verified by disabling the configured LLM endpoint and confirming chat continued to function through the rule-based pipeline with no visible error. The live-fallback latency optimization (Section 5.5) was measured directly: a fallback-triggering query’s total response time fell from 13.6 seconds (re-running full query understanding after ingestion) to 5.8 seconds (retrying with the already-resolved intent/entities passed through).

6.5 Engineering Case Study: Calibrating the Semantic-Trust Threshold

This section documents a real, multi-round bug chain encountered while integrating the optional LLM backend, because the fix illustrates a general principle — distinguishing lexical evidence from semantic correctness — more clearly than an abstract description of the final design would.

Round 1. The original `hybrid_search()` returned the entire catalogue, ranked, whenever a structured filter matched zero candidates — an unconstrained ranking pass over unrelated products presented as search results. Fixed by returning an empty result immediately once a structured filter yields no candidates.

Round 2. Adding the LLM backend reintroduced the bug through a different path: an LLM-supplied category, unlike a rule-based one, is not guaranteed to correspond to anything the message literally said. An initial fix rejected the whole result set whenever every candidate showed zero TF-IDF relevance — but this broke “phone” (the catalogue’s own titles say “Smartphone”, a token mismatch TF-IDF cannot bridge) and every Bangla-script query (an English-only catalogue can never show lexical overlap with Bangla, regardless of correctness).

Round 3. A substring-overlap check (“does any candidate share a substring with the query”) fixed “phone” but not “earphone” (a true synonym, sharing no substring with “Audio” or “earbuds”), and reintroduced the original bug at a larger catalogue size: once the catalogue grew a “Multi Purpose Mixer Grinder Blender Machine” listing, a “washing machine” query — nothing genuine in the catalogue — coincidentally shared the word “machine” with that one Home Appliances product, and because the check operated on the whole candidate set rather than per candidate, that one coincidental match let all 27

unrelated Home Appliances products through uncut.

Final fix. The trust decision was moved to where it belongs — `nlp_llm.py`'s `_category_is_corroborated()` (Section 5.2), checked against the same `CATEGORY_SYNONYMS` dictionary the rule-based pipeline already trusted — combined with a per-candidate, not whole-set, `MIN_RELEVANCE_FLOOR` in `search.py` (Section 5.3). This was verified against every known case simultaneously: “phone”, “earphone”, “headset”, Bangla queries, “washing machine” (correctly empty), “rice cooker” (correctly found once the category-normalization fix in Section 5.5 also landed), “noodles”, and the full Phase-6 evaluation baseline in Section 6.2. The general lesson generalized from this case: a lexical-overlap signal (substring or TF-IDF score) and a semantic-correctness signal (does this category actually apply) are not interchangeable, and conflating them either rejects true positives the lexical signal cannot see, or admits false positives the semantic signal alone would have caught.

CHAPTER 7

Results and System Walkthrough

This chapter presents the working system directly, through screenshots captured from the running application (Laravel + FastAPI, seeded and live-ingested catalogue), rather than mockups.

7.1 Multi-Constraint Search in English

The query “black backpack under 3000 taka with a laptop compartment” is parsed into a category (Bags), a colour (black), and a budget ceiling (3000) simultaneously, and the response states the resolved intent and entities above the ranked results, so the constraint parsing is visible to the user rather than a black box.

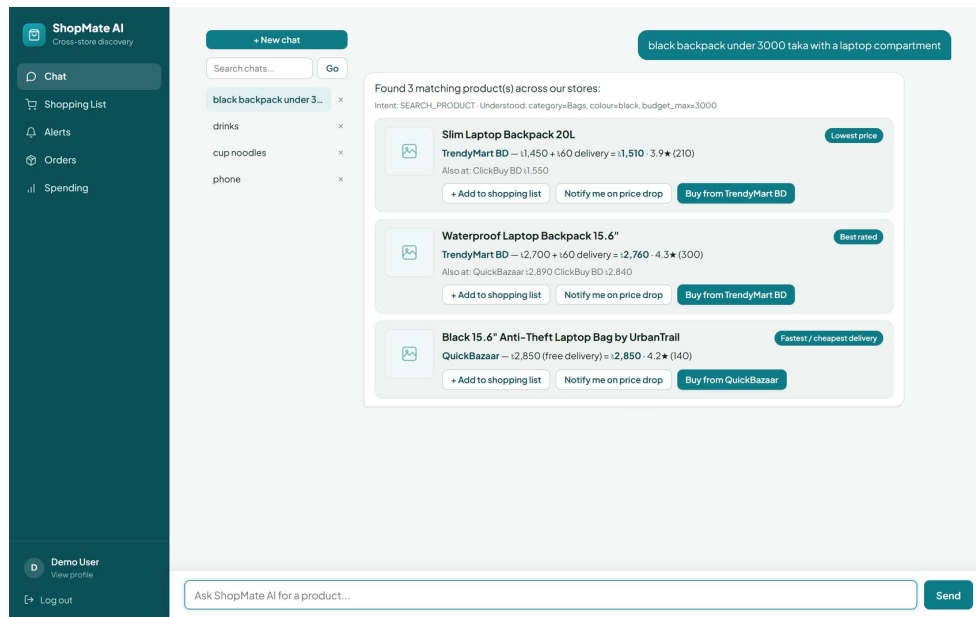


Figure 7.1: English multi-constraint query: category, colour, and budget parsed and applied together.

7.2 The Same Understanding in Bangla

Within the same conversation, the Bangla query কালো ব্যাগ (“black bag”) is understood identically to its English equivalent and returns the same category of results, demonstrating that the entity dictionaries (Section 5.1) and the non-Latin-aware relevance floor (Section 5.3) work together rather than one silently defeating the other.

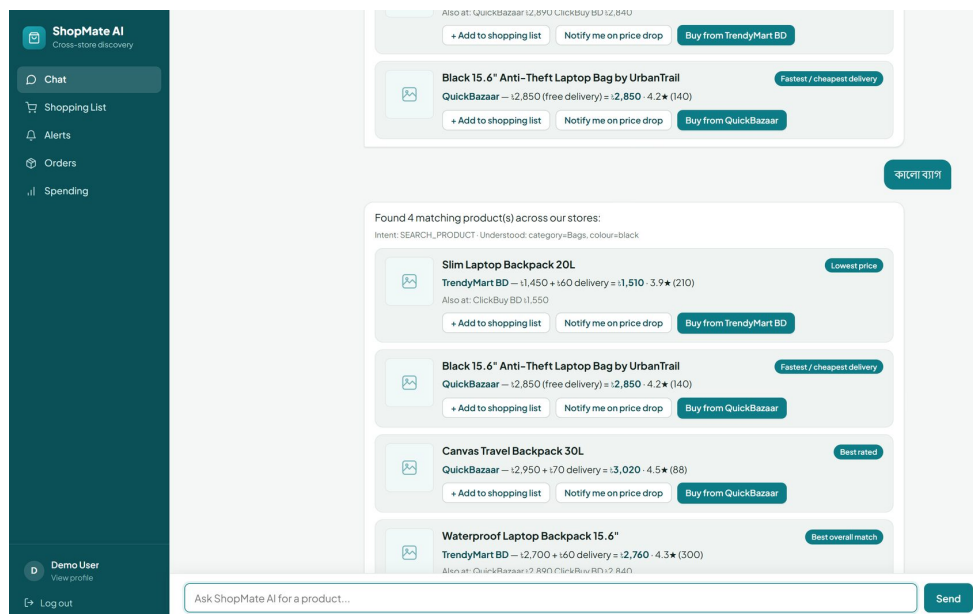


Figure 7.2: Bangla-script query in the same session, understood and answered identically to its English equivalent.

7.3 A Genuinely Absent Product

Searching for “telescope for astronomy” — a product genuinely outside the seeded catalogue and outside what the live-fallback search could find on Othoba.com — correctly reports that nothing is available rather than fabricating or coincidentally matching an unrelated result, and offers the pre-order/notify-me path described in Section 5.6.

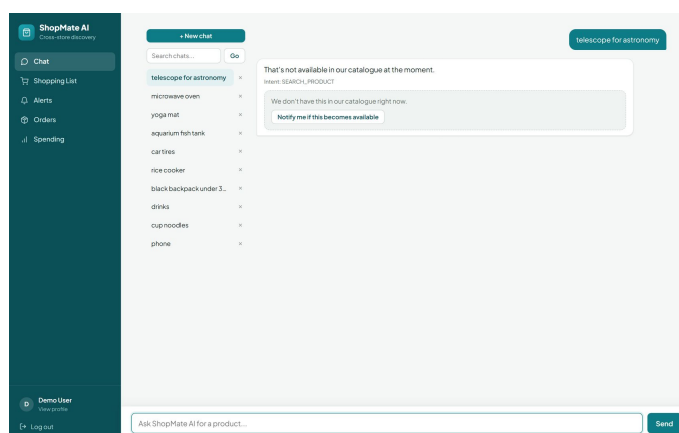


Figure 7.3: A correctly-reported “not available” result, with a pre-order/notify-me option rather than a fabricated match.

7.4 Cross-Store Comparison from a Live-Ingested Product

The order-confirmation screen below is for a rice cooker that ShopMate AI did not have in its seeded catalogue at all — it was found through the live-fallback search described in Section 5.5, correctly categorised as Home Appliances (not Groceries) by the category-normalization fix in the same section, and ingested as a new canonical product through ProductMatchingService. The confirmation step shows the exact listing, its real price and delivery cost, and links out to the store’s own page; no order or payment is submitted anywhere until the user presses “Confirm order” here.

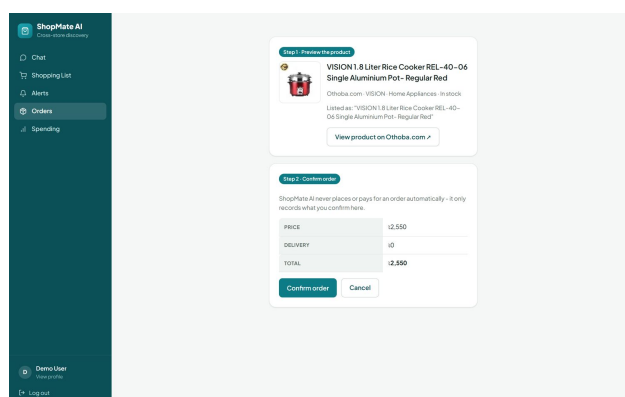


Figure 7.4: Human-confirmed order screen for a rice cooker discovered through the live-fallback search.

7.5 Administrative Dashboard

The admin dashboard reflects live catalogue and operational state — at the time of capture, 284 products across 5 stores, 2 users, 2 recorded orders, 9 logged searches, 0 outstanding pre-order requests, and a mean AI-confidence (ranking score) of 0.729 across recent results. The “Possible Duplicate Products” queue lists pairs the matching service (Section 4.5) flagged as similar but below its auto-merge threshold — visible here are pairs such as two differently-branded Bluetooth earbud listings at 0.50 similarity and two ASUS VivoBook listings at 0.63 similarity, each awaiting a manual “Merge” or “Not a duplicate” decision rather than being auto-merged on a borderline score.

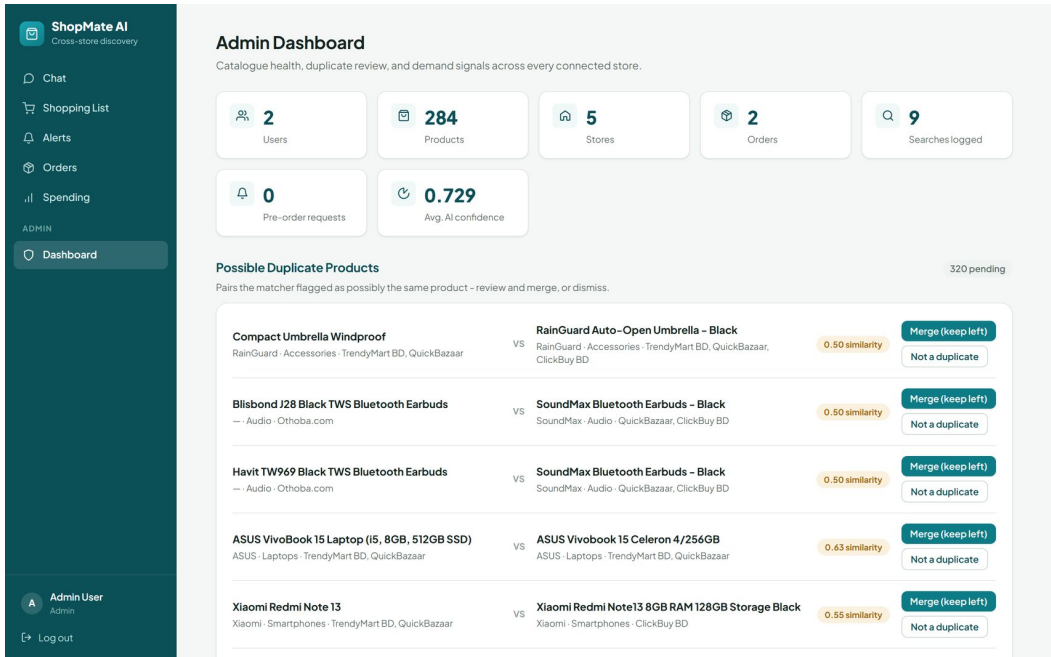


Figure 7.5: Admin dashboard: catalogue health, activity counters, and the manual duplicate-review queue.

7.6 Spending Insights

Beyond the modules in the original proposal (Section 4.3), a Spending Habits view was added during implementation: it aggregates a user's own confirmed orders (Section 5.6) into total spend, order count, average order value, a per-category breakdown, and a six-month trend, with short, data-driven money-management tips generated from that same breakdown (e.g. flagging a shopping-list item with no price-drop alert set).

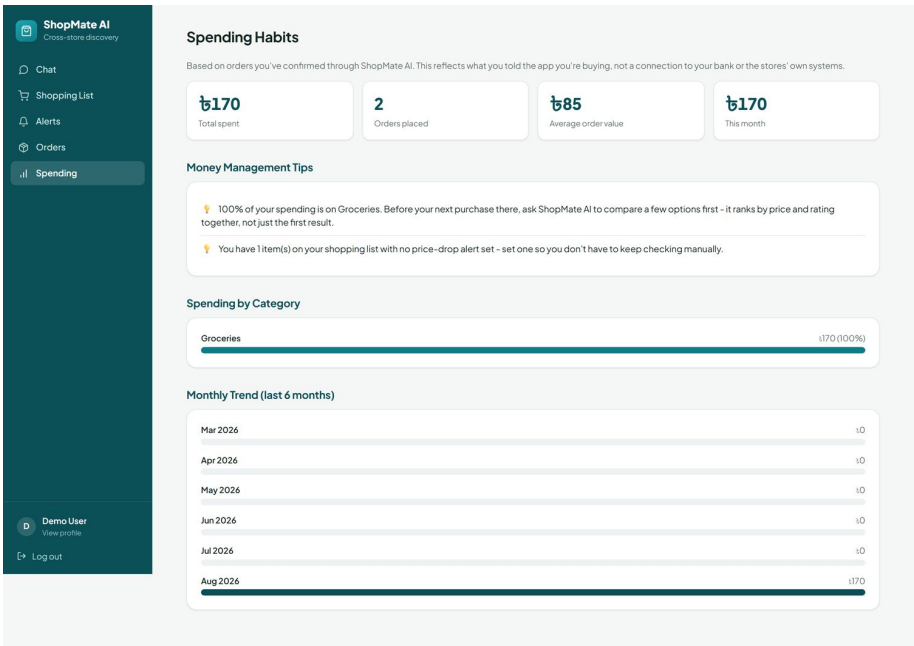


Figure 7.6: Spending Habits view, derived entirely from the user's own confirmed orders.

CHAPTER 8

Conclusion and Future Work

8.1 Summary of Contributions

- A working, service-oriented conversational shopping agent (Laravel + FastAPI) that separates business logic from AI/NLP processing behind one internal API.
- A dual-pipeline query-understanding system — a deterministic rule-based extractor always available, and an optional LLM backend whose every field is independently re-validated against the same closed vocabulary, with unconditional fallback on failure — plus a category-corroboration mechanism that prevents an LLM’s semantic over-reach from silently returning irrelevant results.
- A hybrid TF-IDF and structured-filter retrieval pipeline with a calibrated, per-candidate relevance floor, arrived at through a documented, multi-round debugging process (Section 6.5) that generalizes beyond this project: lexical overlap and semantic correctness are different signals and must be checked separately.
- A dependency-free, two-veto cross-store product-matching service, tuned against both a clean fixture and messy real listing data, that makes genuine multi-store price comparison possible rather than assumed.
- An on-demand live-search fallback that extends catalogue coverage in real time and permanently ingests genuine finds, with a measured latency optimization (13.6s to 5.8s) from skipping a redundant language-understanding round trip on retry.
- A human-confirmed order-recording workflow and a full evaluation against a conventional keyword-search baseline, showing the experimental system reaches Recall@5 = 1.00 and NDCG@5 = 1.00 against the baseline’s 0.59 and 0.59, with the baseline scoring exactly zero on every budget-aware, multi-constraint, and Bangla/Banglish query tested.

8.2 Limitations

- TF-IDF retrieval is a purely lexical signal; it cannot recognise a true synonym pair with no shared substring unless that pair is already known to the CATEGORY_SYNONYMS dictionary, which is why the corroboration and relevance-floor mechanisms in Sections 5.2–5.3 exist as compensating controls rather than a semantic model doing the work directly.
- The evaluation catalogue is a fixed, seeded 20-product set; most queries have only

one to three truly relevant products against a Precision@5 cut-off, which is why Precision@5 is modest for both systems even though ShopMate AI's recall and ranking quality are both perfect on this query set — a larger, production-scale catalogue would let Precision@K discriminate the two systems more sharply.

- Live catalogue coverage depends on one real external integration (Othoba.com); broader live coverage would require additional store adapters behind the existing `StoreProviderInterface` (Section 4.4).
- The cross-store matching threshold (0.65 / 0.80) was tuned against the store data available during this project; a materially different catalogue (different marketing-boilerplate conventions, different brand-naming patterns) would likely need re-tuning or a held-out validation set of its own.
- Purchase actions are intentionally limited to human-confirmed recording rather than automated checkout, by design rather than by omission (Section 1.5) — this is a scope boundary, not a defect, but it does mean the system does not itself complete a transaction.

8.3 Future Work

- Replace or augment TF-IDF retrieval with sentence-transformer embeddings once semantic generalization beyond the synonym dictionary is needed, using the corroboration/relevance-floor pattern from Sections 5.2–5.3 as the template for how to trust a semantic signal safely.
- Add further store-provider integrations behind the existing `StoreProviderInterface` to broaden live and batch catalogue coverage beyond Othoba.com.
- Extend evaluation to a larger, production-scale catalogue and a real user study measuring task completion time and subjective satisfaction, as outlined in the original project proposal's MSc research-evaluation plan.
- Explore a supported, official-API-based order-automation path for stores that offer one, strictly preserving the human-confirmation requirement (Section 5.6) as a design invariant rather than relaxing it.
- Apply active-learning to the `possible_duplicate_products` review queue (Section 7.5): borderline matching decisions an administrator confirms or rejects could retune the Jaccard/threshold parameters in Section 4.5 over time instead of leaving them fixed.

Bibliography

- [1] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013, pp. 3111–3119.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [5] K. Järvelin and J. Kekäläinen, “Cumulated Gain-Based Evaluation of IR Techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, 2002, pp. 422–446.
- [6] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” 2021. <https://owasp.org/www-project-top-ten/>
- [7] Laravel LLC, “Laravel Documentation.” <https://laravel.com/docs>
- [8] FastAPI, “FastAPI Documentation.” <https://fastapi.tiangolo.com>
- [9] scikit-learn developers, “TfidfVectorizer — scikit-learn Documentation.” https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [10] T. Ahmed, “ShopMate AI — Project Proposal: Design and Development of an LLM-Orchestrated, Retrieval-Augmented Conversational Agent for Cross-Platform Product Discovery, Semantic Product Matching and Personalized E-Commerce Recommendation,” Mawlana Bhashani Science and Technology University, 2026.

APPENDIX A

AI Service API Contract

The AI service exposes a single endpoint consumed internally by the Laravel application.

```
POST /chat/query
Content-Type: application/json

{
  "message": "black backpack under 3000 taka with a laptop compartment",
  "include_international": false,
  "intent": null,
  "entities": null
}
```

Response (truncated to one result):

```
{
  "intent": "SEARCH_PRODUCT",
  "entities": {
    "category": "Bags", "brand": null, "colour": "black",
    "budget_max": 3000, "budget_min": null,
    "rating_min": null, "free_delivery": false
  },
  "results": [
    {
      "id": 1, "canonical_title": "Slim Laptop Backpack 20L",
      "category": "Bags", "brand": null, "score": 0.83,
      "reason": "Lowest price",
      "best_offer": {"store_name": "TrendyMart BD", "price": 1450,
        "delivery_charge": 60, "total_cost": 1510,
        "rating": 3.9, "in_stock": true}
    }
  ]
}
```

`intent` and `entities` are accepted on the request specifically to support the live-fallback retry (Section 5.5): when the Laravel controller re-calls this endpoint after ingesting new listings for the same message, it passes back the already-resolved values so the service can skip `understand()` entirely.

APPENDIX B

Evaluation Query Set

Table B.1: Full evaluation query set (ai-service/eval/queries.json), 11 queries against the seeded 20-product catalogue.

ID	Category	Query
budget-1	budget	backpack under 3000 taka
budget-attr-1	budget+attribute	black backpack under 3000 taka with a laptop compartment
brand-1	brand	Xiaomi products
rating-1	attribute (rating)	highly rated wireless earbuds
budget-2	budget	cheap phone under 22000 taka
multi-1	multi-constraint	black smartphone with free delivery under 26000
banglish-1	bangla/banglish	কালো ব্যাগ
banglish-2	bangla/banglish + budget	ব্যাগ ৳3000 er moddhe
ambiguous-1	ambiguous-name	leather office bag
ambiguous-2	ambiguous-name	kettle
ambiguous-3	ambiguous-name	long lasting shoes for running